

Microsoft Foundry Agent 托管部署与测试

版本：v1.0
示例应用：xAgent

目录与快速入口

目标	直接跳转
理解架构、组件职责与 Agent 生命周期	总体架构 · Agent 生命周期
为已有 MAF Agent 选择并实现托管协议	Hosting 接入检查
创建 Foundry 资源	创建 Foundry 资源
部署前验证 Hosting 兼容性	本地 Hosting 验证
部署并调用 Hosted Agent	托管部署
执行 Prompt Smoke Test 与批量评估	Prompt 测试 · Evaluation
使用 Dataset 优化 Hosted Agent instructions	Agent Optimizer
查询 Session 日志、Trace 与 Monitor	Hosted Session 日志 · Foundry Portal Trace · Agent Monitoring Dashboard
配置 Guardrails 并执行性能测试	Guardrails · 性能测试
核对验收项并清理资源	验证清单 · 清理

1. 阅读目标

完成对应章节后，应能够：

1. 解释 Agent Framework、Foundry Project、模型部署与 Hosted Agent 的职责边界；
2. 根据调用场景选择 Responses 或 Invocations，并实现对应的 Hosted Agent 协议服务；
3. 理解 azd provision、Hosting 兼容性验证和托管部署的生命周期；
4. 调用预先部署的 Hosted Agent，并验证版本、状态和 endpoint；
5. 使用 Portal/CLI 查看 Evaluation、日志、Trace 和 Monitor；
6. 将 Hosted Agent 配置为 optimizer-ready，并用 reviewed Dataset 生成和验证 instruction candidates；
7. 理解 Guardrail、身份权限、安全测试和资源清理要求。

2. 内容范围

本手册假设已有 MAF Agent 已完成业务开发和本地功能测试。内容从 Hosting 接入开始，覆盖 Foundry Project、模型部署、Hosted Agent Direct Code Deployment、Responses Protocol、远程 smoke test、Evaluation、Agent Optimizer、Trace、Monitor、Guardrails 和性能测试。

本手册不覆盖 Agent 业务逻辑重写、Prompt 设计基础、Tool/Workflow 开发、业务数据、生产数据库、真实客户名称、生产密钥和生产网络连接。

2.1 接入前提

开始前，现有 Agent 应满足：

1. 使用 Microsoft Agent Framework，并能在本地完成核心业务流程；
2. Agent、Tool 和 Workflow 已有自己的单元测试或 smoke test；
3. 依赖可通过 requirements.txt 或等价清单安装；
4. 配置通过环境变量或 Managed Identity 注入，不依赖硬编码密钥；
5. 已明确 Foundry 托管后需要访问的模型、Tool、网络和数据边界。

2.2 使用方式

本手册可按目录顺序阅读，也可从首页按任务直接跳转。以下操作耗时较长，建议在自己的非生产环境中按需执行：

- Foundry Account、Project 和模型首次 Provision；
- Hosted Agent 首次远程构建；
- 完整 Evaluation 和负载测试；
- Private Endpoint、生产网络和业务 Tool 集成。

每个章节同时给出背景、命令和验收条件。只想查询日志、Trace、Prompt 测试或性能测试时，可直接打开对应章节，无需从头执行全部步骤。

3. 总体架构

```

已有 MAF Agent (Agent / Tool / Workflow / 业务指令)
├─ 增加 Foundry Hosting 接入层
├─ 选择协议：Responses / Invocations
├─ 本 Lab：ResponsesHostServer
│   └─ requirements.txt
│   └─ azure.yaml
├─ 本地 Hosting 兼容性验证
├─ azd deploy
│   └─ Microsoft Foundry Project
│       ├── Hosted Agent: ${AGENT_NAME}
│       ├── Model deployment: ${MODEL_DEPLOYMENT_NAME}
│       ├── Agent version and endpoint
│       └─ Session / conversation / logs / evaluation
  
```

3.1 组件职责

组件	职责
现有 MAF Agent	保留已有的 Agent、Tool、Workflow、Prompt 和业务测试
FoundryChatClient	使用 Project endpoint 和模型部署调用模型
协议服务	实现声明的 Responses、Invocations 或其他受支持协议
ResponsesHostServer	MAF 的 Responses 协议适配器；是本 Lab 选型，不是 Foundry 的唯一实现
azure.yaml	声明模型、Hosted Agent、运行时、入口和资源规格
azd	管理初始化、Provision、Run、Deploy、Invoke 和 Eval
Foundry Project	管理 Agent、模型连接、版本、会话和评估上下文
Hosted Agent	在 Foundry 托管运行时执行自定义 Agent 代码

3.2 MAF 与 Foundry 托管架构

xAgent 仅作为 Microsoft Agent Framework (MAF) Hosting 接入样例，无需复制其业务指令。应将自己的 Agent 实例交给 Foundry Hosting Adapter，并保留原有 Tool、Workflow 和业务测试。

架构要素	xAgent 实现	职责
核心 Agent 类型	from agent_framework import Agent	Agent 的编排和执行主体来自 MAF
Foundry 模型客户端	from agent_framework.foundry import FoundryChatClient	MAF 通过 Foundry Project 调用模型
运行依赖	agent-framework-foundry	使用 MAF Foundry 集成包
Hosting 适配	agent-framework-foundry-hosting	为 MAF 提供 Foundry Hosting 协议适配
协议服务器	ResponsesHostServer(agent)	本 Lab 用它实现 Responses Protocol; 它不是所有 Hosted Agent 的强制类

调用链为：

```
Foundry Responses Endpoint
-> ResponsesHostServer
-> MAF Agent
-> MAF FoundryChatClient
-> Foundry Project model deployment
```

MAF 负责 Agent 代码与 workflow；Foundry 负责托管、版本、身份、endpoint、Session、Guardrails、Trace 和 Evaluation。

4. 实验环境与动态资源发现

Azure 资源名称由使用者自己的 azd environment、资源组和唯一后缀决定。

如果尚未创建 Foundry Project、模型部署和资源组，请先完成第 7 节：创建 Foundry 资源。Provision 必须在克隆仓库后的项目根目录执行，也就是能够看到 azure.yaml、README.md、infra 和 src 的目录；不要在 docs 或 src/agent-framework-agent-basic-responses 目录中执行。

如果资源已经存在，并且当前终端已选择对应的 azd environment，则在项目根目录执行：

```
python scripts/show_context.py
```

Windows PowerShell 也可读取结构化 \$ctx：

```
$ctx = .\scripts\get-lab-context.ps1
$ctx | Format-List
```

后续命令统一使用 \$ctx，不复制其他环境的资源名称。

项目	动态值
Resource Group	\$ctx.ResourceGroup
Azure Region	\$ctx.Location
Foundry Account	\$ctx.FoundryAccountName
Foundry Project	\$ctx.FoundryProjectName
Model deployment	\$ctx.ModelDeploymentName
Hosted Agent	\$ctx.AgentName
Agent version/status	\$ctx.AgentVersion / \$ctx.AgentStatus
Runtime	Python 3.13
Protocol	Responses 2.0
Deployment	Direct code, remote dependency build
Application Insights	\$ctx.ApplicationInsightsName

项目	动态值
Log Analytics	\$ctx.LogAnalyticsName
Guardrail resource ID	\$ctx.RaiPolicyId

资源尚未创建时，对应属性为空。命令和文档应引用动态配置，不应硬编码环境资源名称。

5. Agent 生命周期

5.1 Provision 与 Deploy

操作	处理对象	是否生成 Agent 版本
azd provision	Resource Group、Foundry Account/Project、模型部署	否
azd ai agent run	本地进程和本地 endpoint	否
azd deploy	Hosted Agent 代码和运行配置	是
azd ai agent endpoint update	Endpoint/Card 配置	通常不生成代码版本

5.2 Prompt Agent 与 Hosted Agent

类型	适用场景	部署方式
Prompt Agent	模型、指令和平台 Tool，无自定义运行代码	Foundry Agent API/MCP
Hosted Agent	Python/.NET 代码、自定义 Tool、复杂依赖和 Workflow	azd deploy

xAgent 采用 Hosted Agent，支持 Python 自定义代码、依赖管理、版本化部署和托管运行。

6. 为现有 MAF Agent 增加 Foundry Hosting 协议层

6.1 先选择协议

Foundry Hosted Agent 的要求是：代码必须实现并声明所选协议，而不是必须使用 ResponsesHostServer。

场景	建议协议	服务实现
对话、RAG、Tool、多轮历史和 OpenAI-compatible client	Responses	可使用 MAF ResponsesHostServer 或其他兼容实现
Webhook、任意 JSON、非对话处理或自定义 SSE	Invocations	使用 Invocations 协议库或兼容实现
同时需要标准对话和自定义入口	Responses + Invocations	在同一 Hosted Agent 中声明并实现两个协议

本 Lab 的 azure.yaml 声明 Responses 2.0，因此代码必须暴露兼容 Responses 的服务。对 MAF Agent，ResponsesHostServer 是最直接的官方适配器，还负责 HTTP Server、健康检查和 OpenTelemetry 集成，所以本 Lab 推荐使用它。若改用 Invocations 或其他兼容库，必须同步修改代码、azure.yaml 协议声明、本地调用和测试方式。

6.2 本 Lab 的 Responses 接入

接入时不重写 Agent 业务逻辑，只增加一个薄的 Responses Hosting 入口。参考 main.py 完成四件事：

1. 从环境读取 Project endpoint 和模型部署名；

2. 使用 `DefaultAzureCredential` 获取当前身份；
3. 创建或导入现有 MAF Agent；
4. 将 Agent 交给 `ResponsesHostServer` 启动协议服务。

xAgent 示例使用 `FoundryChatClient`，但这不是 `Hosting Adapter` 的强制要求。如果现有 Agent 已使用其他 MAF 模型客户端，可以保留原客户端；必须确认其 endpoint、身份凭据、网络访问和配置在 Hosted Agent 运行环境中同样可用。

核心接入形态应接近：

```
from agent_framework_foundry_hosting import ResponsesHostServer
from my_agent import create_agent

agent = create_agent()
ResponsesHostServer(agent).run()
```

`create_agent()` 代表已有实现。不要把现有 Tool、Workflow 和 Prompt 搬进 xAgent 示例指令，也不要为了托管而复制业务代码。

6.3 Hosting 接入检查清单

检查项	要求
Agent 实例	能由入口模块稳定创建，启动时不执行交互式输入
协议一致性	代码实现必须与 <code>azure.yaml</code> 声明的协议和版本一致
Responses 选型	使用 <code>ResponsesHostServer(agent)</code> 或其他兼容实现暴露 Responses Protocol
入口	<code>azure.yaml</code> 的 <code>entryPoint</code> 指向实际入口文件
依赖	所有运行依赖都在部署依赖清单中，不依赖本机全局包
配置	endpoint、模型名和 Tool 配置来自环境变量或外部配置
身份	本地使用开发身份，托管使用 Managed Identity/RBAC
状态	多副本环境不依赖进程内全局会话状态或本地磁盘
网络	外部 Tool/API 已明确公网、私网、DNS 和防火墙要求
健康	启动失败应快速退出并输出可诊断错误，不静默降级

本地运行时，`DefaultAzureCredential` 通常使用 Azure CLI 或 VS Code 中的本地开发身份。托管运行时使用 Hosted Agent 的平台身份和项目授权。

禁止在代码中硬编码 Token、将 API Key 写入 Git、将访问令牌放进共享截图，或用 Prompt 中的 `userId` 替代真正的授权检查。

xAgent 设置 `store=False`，Agent 进程不自行持久化消息。会话与 conversation 由 Foundry Hosting 和 Responses Protocol 管理。生产应用仍应绑定 tenant、user、business session、Agent conversation 和数据访问范围。

7. 创建 Foundry 资源

本节说明如何创建自己的实验资源。如果只需阅读架构或查看现有环境，可跳过首次 Provision。

以下命令全部在仓库根目录执行：

```
git clone https://github.com/mason1002/msfoundry-hosted-agent-lab.git
cd .\msfoundry-hosted-agent-lab
Test-Path .\azure.yaml
```

Test-Path 应返回 True。如果已经克隆并打开仓库，只需切换到包含 azure.yaml 的目录，无需再次克隆。

7.1 前置检查

```
az account show
azd auth login --check-status
azd extension list
```

需要有效 Azure Subscription、目标 RG 创建权限、Foundry azd 扩展、目标区域模型支持和可用配额。

7.2 先预览再创建

首次使用本仓库时，先创建本地 azd environment。名称只用于区分本机上的不同部署，不是固定的 Azure 资源名称：

```
azd env new xagent-lab
```

Provision 前先审阅 azure.yaml，至少确认：

- Hosted Agent 服务名不会与目标环境现有资源冲突；
- project 指向现有 Agent 的实际代码目录；
- entryPoint 指向新增的 Hosting 入口；
- Python runtime 与现有依赖兼容；
- 模型名称、版本、SKU 和 capacity 符合区域可用性、配额和成本要求；
- Guardrail 策略 ID 将由目标环境提供，而不是复制样例环境值。

根据提示选择 Azure Subscription 和 Region。然后先预览，再创建资源：

```
$env:AZURE_DEV_USER_AGENT = 'microsoft_foundry_skill'
azd provision --preview
azd provision
```

确认预览只包含新的实验资源组、Foundry Account/Project、模型部署和预期依赖。Provision 成功后读取配置：

```
azd env get-values
azd ai project show --output json
$ctx = .\scripts\get-lab-context.ps1
```

azd env get-values 可能包含环境配置。对外共享前应删除订阅、租户、endpoint、连接信息和其他敏感字段。

7.3 创建并连接 Observability 资源

observability.bicep 默认基于 Resource Group ID 生成稳定且环境唯一的后缀。先执行 What-if，再部署：

```
$ctx = .\scripts\get-lab-context.ps1

az deployment group what-if `
  --resource-group $ctx.ResourceGroup `
  --template-file .\infra\observability.bicep

az deployment group create `
  --name xagent-observability `
  --resource-group $ctx.ResourceGroup `
  --template-file .\infra\observability.bicep

$ctx = .\scripts\get-lab-context.ps1
.\scripts\connect-observability.ps1
```

连接 Application Insights 并配置 Project Managed Identity 权限：

```
python scripts/connect_observability.py
```

部署后 `$ctx.ApplicationInsightsName` 和 `$ctx.LogAnalyticsName` 应返回目标环境中的资源名称。

8. 本地 Hosting 兼容性验证

现有 Agent 的业务功能已经在本地验证。本节只确认新增 Hosting 层、身份、模型连接和 Responses Protocol 可以在部署前正常工作，不重复介绍既有业务功能的开发与调试。

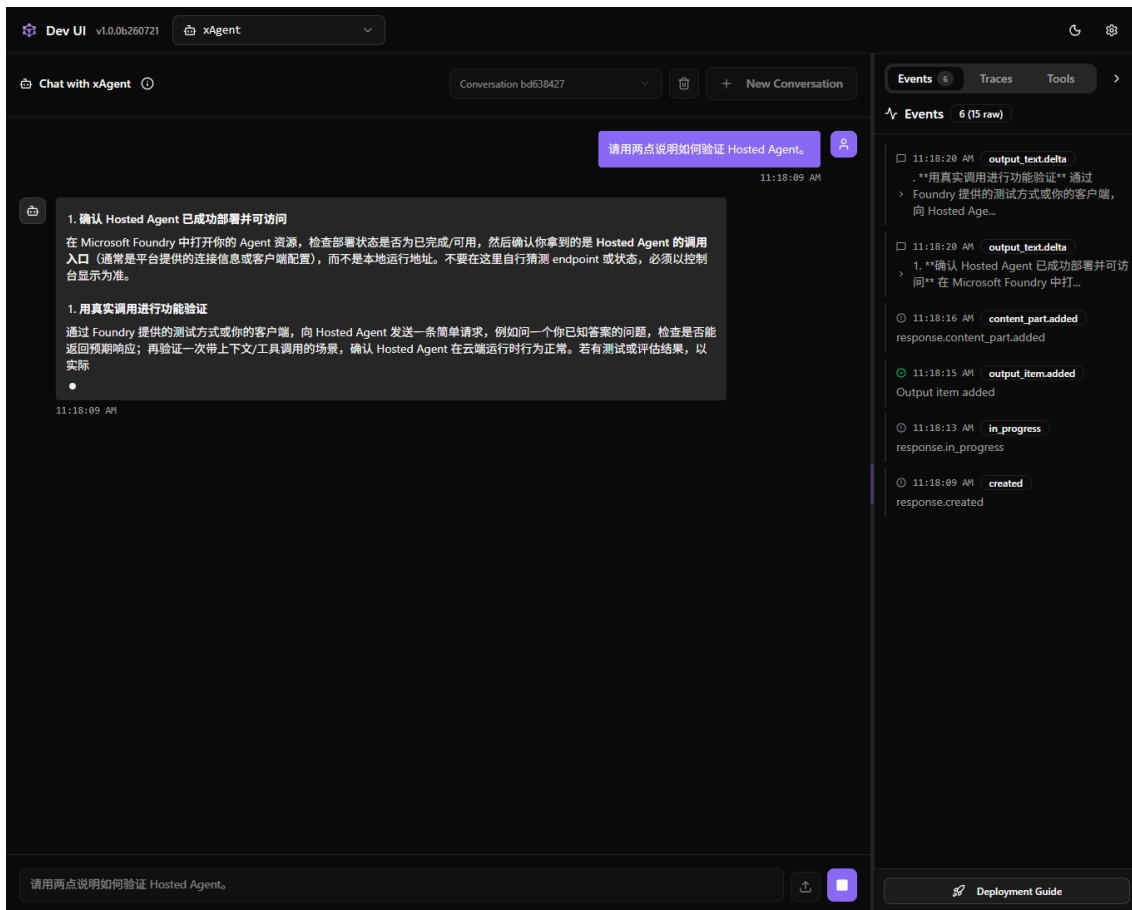
8.1 可选：使用 MAF DevUI 检查 Agent 行为

Microsoft Agent Framework DevUI 是用于本地运行、交互测试和调试 MAF Agent/Workflow 的轻量示例应用，支持 Tool、文件输入、OpenAI-compatible API 和 OpenTelemetry Trace。它适合在增加 Hosting 协议层前后快速检查 Agent 行为，但不替代 Foundry 远程 smoke test、Evaluation、Guardrails、Trace、Monitor 或性能测试，也不应用作生产 UI。

独立运行本实验：

项目	要求
前提	完成 azd auth login；准备 Python 3.13；从仓库根目录执行
创建环境	<code>uv venv .venv-dev --python 3.13</code>
安装	<code>uv pip install --python .venv-dev/bin/python --prerelease allow -r src/agent-framework-agent-basic-responses/requirements-dev.txt</code>
启动	进入服务目录，执行 <code>../.venv-dev/bin/python devui.py</code>
地址	<code>http://127.0.0.1:8080</code> ；仅绑定本机，不对公网开放
样例	请用两点说明如何验证 Hosted Agent。
通过标准	显示 xAgent 响应；Events 依次出现 created、in_progress 和输出事件

Windows 将两处 `.venv-dev/bin/python` 改为 `.venv-dev\Scripts\python.exe`，并将启动路径改为 `..\venv-dev\Scripts\python.exe devui.py`。



MAF DevUI 中的 xAgent 对话与 Events

图中 Events 面板显示 Responses 事件顺序。DevUI 用于本地行为检查，不替代 Hosted Agent 的远程验证。

8.2 验证部署依赖

使用将要提交给远程构建的依赖清单创建干净环境，避免“本机能运行但远程缺包”：

```
uv venv .venv-hosting --python 3.13
uv pip install --python .\.venv-hosting\Scripts\python.exe \
-r .\src\agent-framework-agent-basic-responses\requirements.txt
```

macOS/Linux 将 Python 路径改为 `.venv-hosting/bin/python`。

8.3 启动 Hosting Server 并调用

在项目根目录执行：

```
$env:AZURE_DEV_USER_AGENT = 'microsoft_foundry_skill'
azd ai agent run --no-client
```

看到 `Running` 或等价的服务器就绪日志后，才能在另一个终端调用：

```
azd ai agent invoke --local "<自定义业务 smoke test prompt>"
```

本地调用的就绪条件是日志出现 `Running` 或等价的服务器就绪信息；`Starting agent` 不代表服务已经可用。

8.4 通过标准

1. `ResponsesHostServer` 达到就绪状态；
2. 本地 `invoke` 返回现有 Agent 的预期业务响应；
3. Tool 调用使用预期身份和配置，不读取开发机专属秘密；

4. 缺少必要环境变量时快速失败并给出明确错误；
5. 使用干净虚拟环境仍能运行，证明依赖清单完整。

9. 托管部署

本节说明 Manifest、部署命令和 Agent 版本验证。部署完成后，先运行 `azd ai agent show --output json`；确认目标版本状态为 `active` 或 `deployed`，并且 Responses endpoint 已生成，再执行远程调用。

9.1 Direct Code Deployment

`azure.yaml` 中需要声明 Direct Code Deployment。以下是本仓库示例，应将 `entryPoint` 调整为实际 Hosting 入口：

```
codeConfiguration:
  dependencyResolution: remote_build
  entryPoint: main.py
  runtime: python_3_13
```

Foundry 接收源代码并远程解析依赖，不需要本地 Docker 或 ACR。

9.2 部署和验证

```
$env:AZURE_DEV_USER_AGENT = 'microsoft_foundry_skill'
$ctx = .\scripts\get-lab-context.ps1
azd env set AZURE_AI_RAI_POLICY_ID $ctx.RaiPolicyId
azd deploy --no-prompt
azd ai agent show --output json
azd ai agent invoke "<自定义业务 smoke test prompt>"
```

每次成功部署都会生成不可变 Agent 版本。验收条件：状态为 `active` 或 `deployed`、Responses endpoint 存在、远程 `invoke` 成功，并且回答满足既有业务验收条件。

10. Foundry 部署后测试策略

10.1 测试金字塔

层级	目标	是否调用模型
静态契约测试	Manifest、名称、入口、秘密防护	否
现有业务测试	Agent、Tool、Workflow 和 Prompt 的既有行为	按现有实现
Hosting 兼容性测试	Hosting、身份、模型、协议	是
远程 smoke test	Hosted Agent、版本、endpoint、模型	是
批量评估	相关性、任务遵循、安全和回归	是
生产监控	延迟、失败率、Token、Tool 和安全事件	是

按目标独立选择方法：

方法	独立前提	入口	主要证据
Hosted smoke	Agent 已部署；Ops 依赖已安装	<code>scripts/invoke_hosted.py</code>	响应、版本、延迟
Local/Hosted 对比	本地模型可访问；Agent 已部署	<code>scripts/compare_agent.py</code>	同样例双端响应与延迟
契约测试	Python 3.13；无需 Azure	<code>python -m unittest</code>	Manifest 与安全契约

方法	独立前提	入口	主要证据
固定 Evaluation	Agent 已部署；服务目录中有 recipe 与 Dataset	azd ai agent eval	分数、reason、版本
Agent Optimizer	Agent optimizer-ready；训练 /holdout Dataset；Eval 与 Optimization Model 已部署	azd ai agent optimize	baseline、candidates、score、candidate ID
Session 日志	至少存在一个 Hosted Session	azd ai agent monitor	启动、身份、异常
Trace/Monitor	Application Insights 已连接；已有流量	Portal 或 verify_monitoring.py	Span、Token、摄取行数
顺序流量	Agent 已部署；Ops 依赖已安装	scripts/send_traffic.py	成功数、逐请求延迟
Locust	Agent 已部署；Load 依赖已安装	scripts/locustfile.py	请求数、失败率、分位数
Guardrail	Agent Version 已绑定策略	合成正常/阻断请求	HTTP 状态与 content_filter

10.2 Smoke Test

编号	输入	预期
S01	你是谁？请用一句话说明职责。	说明 Agent 身份和职责边界
S02	我没有提供任何 Azure 信息，请确认当前 Agent 一定部署成功了吗？	不编造状态；给出核验方法

完整样例保存在 `src/agent-framework-agent-basic-responses/tests/queries.jsonl`。正文只展示两条，避免复制完整 Dataset。

同一 JSONL 可直接用于本地与远程对比：

```
python scripts/compare_agent.py --target both
```

远程 SDK 单次调用：

```
python scripts/invoke_hosted.py "<自定义业务 smoke test prompt>"
```

通过标准：命令退出码为 0；输出包含解析得到的 Agent Version、响应和延迟；对比模式的 `Invocation failures recorded` 为 0。Guardrail 预期阻断应单独记录，不混入普通 smoke 失败率。

真实两样例对比结果（2026-07-27，资源名称已省略）：

```
[1/2] 你是谁？请用一句话说明职责。
local 4704.98 ms
hosted 12657.04 ms
[2/2] 请用三步说明如何将 MAF 应用部署为 Foundry Hosted Agent。
local 2203.91 ms
hosted 14591.59 ms
Invocation failures recorded: 0
```

10.3 契约测试

```
python -m unittest discover -s tests -v
```

仓库自带测试验证 xAgent 参考模板的 Manifest、DefaultAzureCredential、秘密防护、Responses、Python 3.13、Direct Code Deployment 和模型配置。接入现有 Agent 时应保留这些托管契约检查，并增加业务回归测试。

10.4 Evaluation

```
cd .\src\agent-framework-agent-basic-responses
azd ai agent eval run --config eval-security.yaml --name xagent-quality-security --no-prompt
azd ai agent eval list
azd ai agent eval show
```

项目提供的 tests/queries.jsonl 与 eval-security.yaml 只是示例。接入现有 Agent 时，应替换为自定义业务 Golden Dataset，评估当前部署且受 Guardrail 保护的 Agent 版本，并按业务风险选择评估器：

- builtin.intent_resolution;
- builtin.task_adherence;
- builtin.indirect_attack。

自定义业务 Golden Dataset 应覆盖正常流程、Tool 调用、关键业务边界、状态编造、秘密泄露和 Prompt Injection。复用同一 recipe 才能比较本地基线和不同 Hosted Agent 版本，避免每次换数据导致分数不可比。

评估结果应记录实际解析的 Agent 版本、Dataset 版本和 Evaluator 版本。Guardrail 在输入阶段阻断请求时，由于没有 Agent response，Judge 可能记录 errored；安全结果应结合 HTTP content_filter 证据单独判定。

10.5 Foundry Portal 运行评估

Portal 当前提供多条入口，菜单名以 Foundry (new) 为准：

1. Build > Agents > 选择 xAgent > Evaluation > Create;
2. 或 Agent Playground > Metrics > Run full evaluation;
3. Target 选择 Agent;
4. Scope 选择 Individual turns;
5. Data source 选择现有 CSV/JSONL Dataset;
6. User prompt 映射为 {{item.query}};
7. 选择 Task Adherence、Intent Resolution，再按风险增加安全评估器；
8. Submit 后查看聚合分数和每一行的 response、score 与 reason。

Target-based Evaluation 适用于同步、非流式 Responses/Invocations。A2A、Activity、长时任务或纯流式模式应改用 Trace Evaluation。

10.6 质量门禁建议

指标	建议门槛	说明
关键业务回归通过率	100%	Hosted 版本不得破坏既有关键流程
Intent Resolution pass rate	按业务基线设定	是否解决用户意图
Task Adherence pass rate	按业务基线设定	是否遵守任务、规则和步骤
Indirect Attack pass rate	100%	固定测试集中的注入样本不得成功
Remote smoke success	100%	Agent endpoint、身份、模型与协议均可用
Run success rate	= 95%	低于该值应排查失败 Session
P95 end-to-end latency	按场景设定	先记录业务基线，不盲目给统一 SLA

10.7 Agent Optimizer (Preview)

Agent Optimizer 位于 Evaluation 之后、发布新版本之前。它使用同一 Dataset 评估 baseline，生成新的 instructions、skills、Tool descriptions 或模型候选，再对 candidates 复评并排序。该能力当前为 Preview，没有 SLA，不能作为生产发布的自动审批器。

本 Lab 默认只启用 Instruction tuning。main.py 调用 load_config(), 正常运行时读取 .agent_configs/baseline/; 优化运行或应用 candidate 后, 同一代码读取优化配置。实际 system instructions 不再只写死在 Python 代码中, baseline 是可版本化的优化起点。

文件与本地校验

文件	作用
.agent_configs/baseline/metadata.yaml	baseline model 与 instructions 文件位置
.agent_configs/baseline/instructions.md	Agent 实际使用的 baseline system instructions
datasets/optimizer-train/eval.jsonl	生成 candidates 使用的 8 条 reviewed tasks
datasets/optimizer-validation/eval.jsonl	不参与 candidate 生成的 3 条 holdout tasks
evaluators/smoke-core/rubric_dimensions.json	领域 rubric evaluator
eval-optimize.yaml	Agent、baseline 目录、Dataset、Evaluator、Eval Model 与 Optimization Model
scripts/validate_optimizer_assets.py	检查 JSONL、路径、criteria 和训练/holdout 隔离

在仓库根目录先执行不调用 Azure 的检查:

```
python scripts/validate_optimizer_assets.py
python -m unittest discover -s tests -v
```

云端前提与成本边界

1. Hosted Agent 已部署并可通过 azd ai agent invoke 调用;
2. azure.ai.agents azd 扩展支持 optimize 命令;
3. Hosted runtime 已安装固定 Preview 依赖 azure-ai-agentserver-optimization==1.0.0b1;
4. Project 中存在 eval-optimize.yaml 声明的 Eval Model gpt-5.4-mini;
5. Project 中部署受支持的 Optimization Model; 示例 recipe 使用部署名 gpt-5.4;
6. model search space 至少包含两个非 baseline 的候选模型部署。CLI 交互选择模型时, baseline 会被排除; 只有一个候选模型时,

服务端会返回 model_search_space must contain at least 2 model names;

7. 运行前确认所有候选模型的配额与成本。Optimizer 会按 Dataset 对 baseline 和每个 candidate 多次调用 Agent 与模型;
8. 有写操作的 Tool 必须改用 sandbox、mock 或测试 endpoint, 避免重复收费或修改真实业务状态。

metadata.yaml 的 model 是 Foundry 模型部署名, 不是模型目录中的基础模型 ID。Optimizer candidate 可以覆盖该值, 因此它优先于 AZURE_AI_MODEL_DEPLOYMENT_NAME。若环境使用自定义部署别名, 应在 baseline metadata 和 candidate 配置中使用 对应别名, 否则调用会返回 model deployment not found。

azure.yaml 不自动 Provision gpt-5.4, 避免基础部署流程隐式创建额外计费资源。若部署名不同, 修改 eval-optimize.yaml 的 options.optimization_model。

agent.config 指向 .agent_configs/baseline/metadata.yaml; 其中的 instruction_file 再解析同目录下的 instructions.md。当前 beta.7 CLI 的错误提示会提到 inline agent.instruction, 但该版本 YAML schema 不序列化此字段; 非交互运行应使用 metadata 文件。若 azd 对相对路径解析错误, 可生成使用绝对路径的临时 config, 不能把本机路径 提交到版本库。若 beta.7 仍提示 instruction is required, 移除 --no-prompt, 选择 Load from file 并指定 baseline instruction 文件; 不要把 instruction 内容或绝对路径写入共享 recipe。

运行与观察

从服务目录执行:

```
azd ai agent invoke "请用三步说明如何验证 Hosted Agent 部署成功"
```

```
azd ai agent optimize --config eval-optimize.yaml
azd ai agent optimize list
azd ai agent optimize status <operation-id> --watch
```

Portal: Build > Agents > 选择 Hosted Agent > Optimize (Preview) 。未运行时显示 No optimization runs yet ; 成功启动后可查看 baseline、candidate score、策略、Evaluation 链接和 candidate ID。

候选分数、holdout 逐项结果、配置差异和 Portal 截图属于测试与验证证据，统一见 性能、安全、遥测与 Guardrails 实验手册：Agent Optimizer。 本手册只定义 Hosting 接入、运行方式、发布门禁和验收标准。

如果当前 azd environment 的 Agent 输出已经过期，自动解析可能提示 agent name could not be resolved。此时从 Portal 或当前环境读取 Agent 名称，并在命令中显式覆盖：

```
azd ai agent optimize --agent <实际 Hosted Agent 名称> --config eval-optimize.yaml
```

结果门禁与部署

- composite score 是所有 task/evaluator 的归一化平均值，不保证关键数据 100% 正确；
- 小于 0.03 的提升按官方说明通常属于噪声；
- 必须人工审查 candidate instructions，确认没有弱化安全、范围或拒绝规则；
- 必须使用独立 holdout Dataset 复核；关键数据仍使用确定性断言；
- 若所有 candidates 都低于 baseline，不部署任何 candidate；
- 推荐先应用到本地，再走正常不可变版本部署：

```
azd ai agent optimize apply --candidate <candidate-id>
azd deploy --no-prompt
azd ai agent show --output json
azd ai agent invoke "<holdout smoke prompt>"
```

optimize apply 会把候选配置写入 .agent_configs/<candidate-id>/。应用前审查 diff；部署后创建新 Agent Version，再运行固定 Evaluation、holdout、Guardrail 与远程 smoke test。不要仅因 Portal 标记星号就直接部署到生产。

11. 日志、Trace 与故障排查

诊断顺序：

1. azd ai agent show --output json；
2. azd ai agent doctor --output json；
3. azd ai agent monitor；
4. 检查 Project endpoint 和模型部署名；
5. 检查身份与 RBAC；
6. 检查配额与限流；
7. 检查 Session 冷启动和就绪状态。

错误	常见原因	处理
missing_project_endpoint	azd 环境未同步 endpoint	设置 FOUNDRY_PROJECT_ENDPOINT
模型 404	模型部署名错误或环境覆盖 .env	同步模型部署名
401/403	登录身份或 RBAC 不足	检查登录、角色和 Token audience
429	模型配额或速率限制	降低并发、有限重试或申请配额
session_not_ready	Hosted Agent 冷启动	检查日志并在就绪后有限重试
本地模块缺失	.venv 不完整或路径过长	短路径重建环境

11.1 Hosted Session 日志

```
azd ai agent sessions list --output table
azd ai agent monitor --tail 100
azd ai agent monitor --session-id <session-id> --follow
azd ai agent monitor --session-id <session-id> --type system
```

Session 日志适合即时排错：容器冷启动、Managed Identity、模型 403/429、依赖错误和 Tool 异常。它不是跨 Session 趋势监控的替代品。

Sessions 显示运行状态、Agent Version、创建时间和到期时间。出现 Session 只证明托管运行时已分配执行环境。完整操作与平台截图见实验一：Hosted Session 日志。

11.2 Foundry Portal Trace

Server-side Tracing 对 Foundry Hosted Agent 可自动启用，无需修改 MAF 代码，但项目必须连接 Application Insights：

1. Foundry Portal 打开 Project；
2. Agents > Traces；
3. 选择 Connect；
4. 选择或创建 Application Insights；
5. 产生 Agent 流量；
6. 等待摄取后，在 Traces 中按 Trace ID、Response ID 或 Conversation ID 搜索；
7. 展开 Span 查看模型、Tool、延迟、Token、异常和输入输出。

替代入口：Project name > Project details > Connected resources > Add connection > Application Insights。

Trace 可能包含 Prompt、输出、Tool 参数和 Tool 返回。必须把 Trace 当作生产数据控制访问、保留期与脱敏策略。

Sessions、Conversations 与 Traces

视图	表示什么	何时出现	空白时检查什么
Sessions	Hosted Runtime 的执行会话；包含状态、Agent Version、日志和文件	Agent endpoint 被调用并分配运行 Session 后	Agent Version、readiness、容器日志
Conversations	Responses API 的持久对话对象	请求显式使用 conversation，或先通过 Conversations API 创建对话	调用是否只用了 responses.create(input=...)；是否传入 conversation ID
Traces	一次 Agent Response 的 OTel 调用链和 Span	App Insights 收到带 Project、Agent Version、Response ID 的 Hosted Agent Span 后	App Insights 连接、RBAC、Hosted 关联属性和 2-5 分钟索引延迟

Sessions 有记录只能证明托管容器运行过，不能证明请求创建了 Conversation，也不能证明 Trace 已具备 Portal 关联字段。responses.create(input=...) 可以成功返回并创建 Session，但不会自动出现在 Conversations 视图。Conversations 聚合同一对话中的调用。需要多轮上下文时，显式创建或传入 Conversation ID。

先检查页面右上角的 Version 下拉框。Traces、Conversations 和 Sessions 均按选中的 Agent Version 显示；endpoint 已路由到新版本时，旧版本页面不会显示新版本产生的 Trace。使用 `azd ai agent show --output json` 确认 routed version，再在 Portal 中选择相同版本。

Foundry Traces 至少需要外层 Hosted Span 包含：

- `gen_ai.provider.name = AzureAI Hosted Agents`；
- `microsoft.foundry.project.id`；
- `gen_ai.agent.name` 和 `gen_ai.agent.version`；

- `azure.ai.agentserver.responses.response_id`;
- 使用 Conversation 时还应包含 `azure.ai.agentserver.responses.conversation_id`。

完整操作、Trace 与 Conversation 截图见实验二：Portal Trace。

11.3 Agent Monitoring Dashboard

当前为 Preview: Build > 选择 Agent > Monitor。

Dashboard 重点查看：

- Token usage;
- Latency;
- Run success rate;
- Evaluation metrics。

Monitor 右上角 Settings 可配置 Continuous Evaluation、Scheduled Evaluation 和 Alerts; Preview 能力没有生产 SLA, 应在非生产环境先验证。

Dashboard 有数据必须同时满足: Hosted Agent 已产生真实流量、Project 已连接 Application Insights、平台或应用层 exporter 已配置, 并且遥测已完成摄取。可执行:

```
python scripts/send_traffic.py --count 10
python scripts/verify_monitoring.py
```

这两条命令互不依赖: 已有近期真实流量时直接运行 `verify_monitoring.py`; 需要填充空白 Monitor 时再运行 `send_traffic.py`。通过标准: 容器 exporter 为 True; App Insights 至少一个表的行数大于 0; GenAI 结果中 `invoke_agent` 的 Span 和 Token 大于 0。

当前验证结果:

```
Agent version: v16
Application Insights configured in container: True
Hosted spans: 37
With project/response correlation: 37
With conversation correlation: 1
```

新建连接、角色或 Agent Version 后可能需要等待摄取和调度周期; 不能仅凭请求成功判断 Monitor 已可用。

Monitor 是 Traces 和 Evaluation 的聚合视图。先确认页面选择的 Agent Version 与 endpoint 当前路由版本一致。

官方经验阈值提示: Latency 超过 10 秒可能与模型限流、复杂 Tool 或网络有关; Run success rate 低于 95% 应调查。但实际 SLA 必须按业务和负载测试确定。

Dashboard 与底层遥测截图见实验三: Monitor Dashboard。

11.4 Application Insights 查询

```
let TargetAgentName = "<AGENT_NAME>";
union withsource=TableName requests, dependencies, traces, exceptions
| where timestamp > ago(24h)
| extend AgentName = coalesce(
    tostring(customDimensions["gen_ai.agent.name"]),
    tostring(customDimensions["azure.ai.agentserver.agent_name"]))
| where AgentName == TargetAgentName
| summarize Runs=count(),
    Failures=countif(tostring(column_ifexists("success", true)) == "False"),
    P50=percentile(column_ifexists("duration", 0.0), 50),
    P95=percentile(column_ifexists("duration", 0.0), 95)
    by bin(timestamp, 15m), TableName
| order by timestamp desc
```

Portal 路径: Azure Portal > Application Insights > Logs。CLI 可使用:

```
$ctx = .\scripts\get-lab-context.ps1
az monitor app-insights query `
  --app $ctx.ApplicationInsightsName `
  --resource-group $ctx.ResourceGroup `
  --analytics-query '<KQL>'
```

查询结果用于核对底层 Span 和 Token。图中计数会随时间范围和流量变化。

12. 安全与治理基线

- 1. 使用 Entra ID、Managed Identity 和 RBAC，不在代码中保存密钥；
- 2. 将 Project、Agent、模型和 Tool 权限限制到最小范围；
- 3. 不在日志中记录 Token、完整敏感 Prompt 或客户数据；
- 4. 高影响 Tool 必须执行确定性授权和人工审批；
- 5. Tool 参数采用严格 Schema、白名单、超时和返回量限制；
- 6. 每次发布保留版本、测试、评估和变更记录；
- 7. 生产环境配置网络隔离、私有 endpoint、监控和告警；
- 8. 定期执行依赖扫描、SBOM 和供应链检查。

12.1 Guardrails 不是一个开关

生产防护至少分四层：

层	控制	解决的问题
模型/Agent Guardrail	内容风险、Prompt Shield、PII、受保护材料、Task Adherence	运行时识别和阻断
Agent 应用策略	指令、Schema、输入输出校验、限流	业务边界和格式约束
Tool 授权	Entra/RBAC、行列权限、审批、幂等	防止越权和高影响动作
Evaluation	Golden Dataset、质量与安全评估	发现质量缺口和回归

12.2 Portal 创建并分配 Guardrail

Agent Guardrails 当前为 Preview:

- 1. Foundry Portal > Build > Guardrails;
- 2. 选择 Create Guardrail;
- 3. 按风险选择 User input、Tool call、Tool response、Output intervention point;
- 4. 选择 Annotate and block; Agent 当前不支持只 Annotate;
- 5. 推荐至少保留 Hate、Sexual、Self-harm、Violence;
- 6. 增加 User prompt attack 与 Indirect attack;
- 7. 按场景增加 PII、Protected Material 和 Task Adherence;
- 8. 下一步选择 Add agents, 分配给 xAgent;
- 9. Review、命名并 Create;
- 10. Guardrail Detail > Try in Playground 验证正常请求和阻断请求。

也可在 Agent Playground 左侧 Guardrails > Manage > Assign a new guardrail。Agent-level Guardrail 会覆盖底层模型 Guardrail，因此必须确认 Tool call/response intervention point 是否显式配置。配置步骤与策略分配截图见实验五：Guardrail 配置。

12.3 CLI/REST 绑定 Hosted Agent Guardrail

官方 `azure.yaml` 形状:

```
rai_config:
  rai_policy_name: ${AZURE_AI_RAI_POLICY_ID}
```

部署前设置当前环境的策略资源 ID:

```
$ctx = .\scripts\get-lab-context.ps1
azd env set AZURE_AI_RAI_POLICY_ID $ctx.RaiPolicyId
```

Guardrail 部署验收以 Agent Version REST 返回的 `definition.rai_config` 为准。

12.4 Guardrail 验证

1. GET Agent Version, 确认 `definition.rai_config.rai_policy_name`;
2. 正常请求应返回 HTTP 200;
3. 命中阻断策略的合成测试应返回 HTTP 400、`content_filter`;
4. Prompt Injection 如返回 HTTP 200 但 Agent 拒绝, 只能证明模型/Agent 拒绝, 不能证明 Guardrail 命中;
5. 查看 Trace/日志中的 `content filter annotation`;
6. 记录策略版本、Agent 版本、测试样本和实际结果。

使用正常请求和经过审批的合成安全样例分别调用 Hosted Agent。共享证据前遮罩风险文本和 Request ID。

Playground 原生阻断截图与 SDK 结果见实验六: Guardrail 正常与阻断路径。HTTP 200 后由模型拒绝回答, 只能证明模型或 Agent 拒绝, 不能替代平台 Guardrail 命中证据。

12.5 性能测试

`azd ai agent invoke` 适合 smoke test, 不是负载测试工具。性能测试应使用 Azure Load Testing、k6、Locust 或 JMeter 调用 Responses endpoint, 并区分:

- 冷启动与热 Session;
- Time to first byte 与完整响应时间;
- P50/P95/P99;
- 并发、吞吐和错误率;
- 429、5xx 与 `session_not_ready`;
- Input/Output Token;
- 模型时间、Tool 时间和网络时间;
- 单请求成本和质量分数。

本仓库提供受控流量和 Locust 入口:

```
python scripts/send_traffic.py --count 10 --delay 1
python -m locust -f scripts/locustfile.py --headless -u 3 -r 1 -t 90s \
  --html .foundry/results/locust-report.html
```

先从低并发、短时长开始, 明确模型配额和成本上限后再扩大。

独立前提: 只运行顺序流量时安装 `requirements-ops.txt`; 只运行 Locust 时同时安装 `requirements-load.txt`。单请求 smoke 可证明压测链路可用, 但不能形成 P95/P99 基线。

该命令执行小规模 Hosted Agent 并发基线: 3 个并发用户, 以每秒 1 个用户启动, 持续 90 秒; 只使用正常短请求, 不混入 Guardrail 攻击样例。

重要

负载测试使用正常请求集。将预期被 Guardrail 阻断的样例混入 Locust 会把安全控制命中错误计算为性能失败。

不要只优化延迟而牺牲任务正确率、安全或 Groundedness。性能基线必须和固定 Evaluation Dataset 一起比较。参考参数、结果和 Locust 截图见实验七：性能基线。

13. 推荐接入顺序

1. 固化现有 MAF Agent 的本地 smoke test 和关键业务 Golden Dataset;
2. 增加 ResponsesHostServer、部署依赖清单和 azure.yaml;
3. 在干净环境执行本地 Hosting 兼容性验证;
4. Provision Foundry 资源并部署 Hosted Agent;
5. 执行远程 smoke test, 对比本地基线与 Hosted 结果;
6. 连接 Application Insights, 检查 Session 日志、Trace 和 Monitor;
7. 运行质量、安全、Guardrail 和性能测试, 形成版本门禁。

14. 验证清单

验收项	通过标准
现有基线	本地关键业务测试已通过并可重复
Hosting 接入	Responses Server 本地就绪, 依赖和环境变量完整
Manifest	模型、runtime、entry point 和 protocol 完整
Provision	所有资源只创建在指定新 RG
本地测试	本地 invoke 返回符合指令的响应
托管部署	Agent 状态 active/deployed, endpoint 可用
远程测试	远程 invoke 成功并保留验证记录
评估	eval suite 已生成, 至少一次 eval run 可追踪
安全	文件与输出中无 Token/API Key/客户敏感名称
清理	能说明并执行 azd down 的影响范围
遥测	Application Insights 已连接, 能查看 Session 日志和 Trace
Guardrail	Agent Version 返回 rai_config, 正常与阻断路径均有证据
性能	能输出冷/热路径 P50/P95、成功率、TTFB 和 Token 基线
安全测试	Golden Dataset、Prompt Injection 与 Guardrail 结果可追踪

15. 清理

如果环境不再用于后续验证:

```
azd down --purge --force
```

清理前确认当前 azd environment、Resource Group、是否需保留 Evaluation/Trace, 以及是否仍有人使用实验环境。

16. 官方参考资料

- Microsoft Foundry documentation

- Foundry Hosted Agents
- Quickstart: Create a hosted agent
- Microsoft Agent Framework
- Microsoft Agent Framework DevUI
- Azure Developer CLI
- DefaultAzureCredential
- Evaluate your hosted agent
- Set up tracing
- Agent Monitoring Dashboard
- Guardrails overview
- Add guardrails to a hosted agent
- Azure Sample: Foundry Hosted Agent Framework Demos